

Ansible_Play_Books:

- Ansible playbooks are ****YAML files**** that define automation tasks (like configuration, deployment, orchestration) to be executed on remote hosts.
- They are the ****core of Ansible automation****, allowing you to describe desired states and consistently enforce them across environments.

Definition: A playbook is a ****blueprint of automation tasks**** written in YAML. It tells Ansible **what to do** and **where to do it**.

- **Structure:**
- **Plays:** Each play targets a group of hosts.
- **Tasks:** Ordered steps (e.g., install packages, copy files).
- **Modules:** The actual units of work (e.g., `yum`, `copy`, `service`).
- **Variables:** Allow customization and reusability.
- **Handlers:** Triggered by tasks (e.g., restart a service after config change).
- **Roles:** Collections of tasks, variables, and files for modular automation.

Why Use Playbooks Instead of Ad-hoc Commands?

- **Repeatability:** Define once, run many times with consistent results.
- **Scalability:** Apply the same workflow across hundreds of servers.
- **Version Control:** YAML files can be tracked in Git.
- **Complex Logic:** Support for conditionals, loops, and error handling.

Example of a Simple Playbook

```
``yaml
- name: Install and start Apache

hosts: webservers

become: yes

tasks:

  - name: Install Apache

    yum:

      name: httpd

      state: present
```

```
- name: Start Apache service

service:

  name: httpd

  state: started

'''
```

👉 This playbook installs Apache (`httpd`) and ensures the service is running on all servers in the `webserver` group.

Best Practices

Keep playbooks modular → Use roles for reusability.

Validate syntax → Run `ansible-playbook --syntax-check`.

Test incrementally → Start small before scaling.

Use variables and templates → Avoid hardcoding values.

Ensure idempotence** → Tasks should not cause changes if the system is already in the desired state.

In short: Playbooks are the ****heart of Ansible automation****, turning manual steps into reproducible, scalable workflows. They let you manage everything from a single server to thousands of nodes with clarity and control.

****IMPORTANT:****

Only the space character can be used for indentation; tab characters are not allowed.

If you use the **vi** text editor, you can apply some settings which might make it easier to edit your playbooks. For example, you can add the following line to your **\$HOME/.vimrc** file, and when **vi** detects that you are editing a YAML file, it performs a 2-space indentation when you press the Tab key and auto indents subsequent lines.

```
```bash

autocmd FileType yaml setlocal ai ts=2 sw=2 et

'''
```

## Essential Ansible Playbook Commands

### 1. Run a Playbook

```
```bash
```

```
ansible-playbook site.yml
```

```
```
```

- **\*\*Purpose:\*\*** Executes the playbook (`site.yml`) on the defined inventory.
- **\*\*Most common command\*\*** for applying automation.

### 2. Syntax Check

```
```bash
```

```
ansible-playbook site.yml --syntax-check
```

```
```
```

- **\*\*Purpose:\*\*** Validates YAML syntax before running.
- **\*\*Best practice:\*\*** Always check syntax to avoid runtime errors.

### 3. Limit Hosts

```
```bash
```

```
ansible-playbook site.yml --limit webservers
```

```
```
```

- **\*\*Purpose:\*\*** Restrict execution to a specific group or host.
- Useful for **\*\*targeted testing\*\***.

### 4. List Hosts

```
```bash
```

```
ansible-playbook site.yml --list-hosts
```

```
```
```

- **\*\*Purpose:\*\*** Shows which hosts will be affected.
- Helps confirm **\*\*inventory scope\*\***.

## 5. List Tasks

```
```bash
```

```
ansible-playbook site.yml --list-tasks
```

```
```
```

- **\*\*Purpose:\*\*** Displays all tasks in the playbook without running them.
- Great for **\*\*reviewing workflow\*\***.

## 6. Dry Run (Check Mode)

```
```bash
```

```
ansible-playbook site.yml --check
```

```
```
```

- **\*\*Purpose:\*\*** Simulates execution without making changes.
- Ensures **\*\*idempotence\*\*** and safety before applying.

## 7. Start at Specific Task

```
```bash
```

```
ansible-playbook site.yml --start-at-task="Install Apache"
```

```
```
```

**\*\*Purpose:\*\*** Resume execution from a specific task.

Saves time when troubleshooting.

## 8. Tags

```
```bash
```

```
ansible-playbook site.yml --tags "setup"
```

```
ansible-playbook site.yml --skip-tags "debug"
```

```
```
```

- **\*\*Purpose:\*\*** Run or skip specific tagged tasks.
- Enables **\*\*selective execution\*\***.

## 9. Verbose Mode

```
```bash
```

```
ansible-playbook site.yml -v
```

```
ansible-playbook site.yml -vvv
```

```
```
```

- **Purpose:** Increase output detail.
- Useful for **debugging**.

## 10. Vault Integration

```
```bash
```

```
ansible-playbook site.yml --ask-vault-pass
```

```
```
```

- **Purpose:** Run playbooks with encrypted secrets.
- Ensures **secure automation**.

## Pro Tips for Engineers Like You

- **Combine** `--check` with `--diff` to preview file changes.

- Use **extra-vars** (`-e`) for dynamic variables:

```
```bash
```

```
ansible-playbook site.yml -e "env=production"
```

```
```
```

- Always **test on a subset of hosts** before scaling to production.
- Integrate with **CI/CD pipelines** for automated validation.

## PlayBook Examples:

### Example 1: Install a Package

```
``yaml
- name: Install Nginx on webserver

 hosts: webserver

 become: yes

 tasks:

 - name: Ensure Nginx is installed

 yum:

 name: nginx

 state: present
...

```

👉 This playbook installs **\*\*Nginx\*\*** on all hosts in the `webserver` group.

### Example 2: Copy a File

```
``yaml
- name: Deploy configuration file

 hosts: all

 become: yes

 tasks:

 - name: Copy custom config

 copy:

 src: /home/user/config.conf

 dest: /etc/myapp/config.conf

 owner: root

 group: root

 mode: '0644'

```

...

👉 Copies a local file to remote hosts with correct permissions.

### Example 3: Start a Service

```
``yaml
```

```
- name: Ensure Apache is running
```

```
 hosts: webservers
```

```
 become: yes
```

```
 tasks:
```

```
 - name: Start Apache service
```

```
 service:
```

```
 name: httpd
```

```
 state: started
```

...

👉 Makes sure **Apache** is running on target servers.

### Example 4: Create a User

```
``yaml
```

```
- name: Create a new user
```

```
 hosts: all
```

```
 become: yes
```

```
 tasks:
```

```
 - name: Add user 'deploy'
```

```
 user:
```

```
 name: deploy
```

```
 state: present
```

```
 shell: /bin/bash
```

👉 Adds a `deploy` user across all hosts.

### Example 5: Using Handlers

```
``yaml
- name: Update Apache config
 hosts: webservers
 become: yes
 tasks:
 - name: Copy Apache config
 copy:
 src: apache.conf
 dest: /etc/httpd/conf/httpd.conf
 notify: Restart Apache
 handlers:
 - name: Restart Apache
 service:
 name: httpd
 state: restarted
...

```

👉 If the config changes, the handler restarts Apache.

